

Defense guide: chunk-recall-audit

For reading before an interview. Every number in this file is re-measured by `tools/collect_metrics.py` and checked against this text by `tools/check_numbers.py`, so a sentence that has gone stale fails the build rather than getting read aloud.

Read it in three passes. The first two sections are what to say in the first minute. The claims table and the sections under it are what to say when someone picks one. The last three sections are what to say when someone attacks it, which is the part worth rehearsing.

The thirty second version

"Recall at k asks whether any retrieved chunk touches the answer, so a chunk holding the last three words of it counts. The question anyone actually cares about is whether one single chunk held enough to answer, and whether a chunk holds an answer whole is a question about character offsets with an exact answer and no model in it. Aggregate that over a labelled corpus and you get a ceiling that bounds every retriever, including an oracle. On 200 character **chunks the ceiling is 0.2917** while the standard metric on the same rankings reports 1.0."

Then stop. The question that usually comes next is "so what do I do about it", and the answer is the failure attribution: across **13 chunkings** there were **1663 failures**, **1661 were the chunker's** and **2 were the retriever's**, so on this corpus an embedding upgrade has a measured return of nothing.

The two minute version

The structural claim first, because it does not depend on my corpus. A retrieval metric is a function of the retrieved chunks, so it cannot distinguish a chunk that never held the answer from a chunk that held it and was not returned. Containment is a different function, of the chunking alone, and it is decidable by interval arithmetic on character offsets before any retriever exists. That gives a bound rather than a score, and a bound is the more useful object: it tells a team the best outcome available on their current chunking, which is the number they need before they choose a retriever.

Then the measurement, because the structural claim does not tell anyone how much it costs them. The worst configuration in the plan **reports a recall of 1.0** while the pipeline **actually answers 0.2528**, and **269 of them cannot answer** on their own, **a gap of 0.7472**. Span complete recall is **0.475 at four hundred** characters and **0.7389 at eight hundred**, against **a gap of 0.7056** at two hundred. And **10 of the thirteen** configurations have no retriever fixable failure at all.

Then the number I would lead with if I only had one. The gap does not close by retrieving more: it is **0.85 at k equal to one** and **still 0.7472 at twenty**, because span complete recall goes **flat from k equal to 3** while the standard metric climbs to one. Retrieving more copies of a broken answer does not assemble one.

The claims, and how each one is proved

Claim	Command	The number that settles it
The two metrics are far apart, and only one means what you think	python experiments/exp01_the_gap_the_standard_metric_hides.py	1.0 reported against 0.2528 actual on the same rankings
Almost nobody's retrieval problem is a retrieval problem	python experiments/exp02_who_can_fix_this.py	1661 of 1663 failures were the chunker's
The overlap is a guarantee threshold, and it is exact	python experiments/exp03_the_guarantee_threshold.py	101 characters, verified by exhaustion over every position
The ceiling bounds every retriever, and it is tight	python experiments/exp05_which_lever_matters.py	an oracle attains it with a worst difference of 0.0
The chunker is the bigger lever	the same experiment	0.7917 against 0.1972, four times as far
The bound is cheap enough that not knowing it has no excuse	make bench	2.2 ms against 50.1 ms at the shipped corpus size

The two metrics are far apart

The **worst is fixed-200-150**, where the standard metric **reports a recall of 1.0**, the pipeline **actually answers 0.2528**, and **269 of them cannot answer** on their own out of **360 questions in 120 generated documents**. On the default 200 character **chunks the ceiling is 0.2917**, leaving a **gap of 0.7056**, and span complete recall is **0.475 at four hundred** and **0.7389 at eight hundred**.

If pushed on "that is a synthetic corpus, so the gap is synthetic": the gap's size is, and its *existence* is not. Span complete recall is bounded by the ceiling and standard recall at k is not bounded by anything, so on any corpus where answers are longer than the chunks the two must diverge. The corpus decides how much.

Almost nobody's retrieval problem is a retrieval problem

Across **13 chunkings** at k equal to 5, BM25 produced **1663 failures**: **1661 were the chunker's** and **2 were the retriever's**. **10 of the thirteen** configurations have no retriever fixable failure at all. The hashed vector retriever, which is materially weaker, **produced 177 retrieval failures** against the same 1661 from chunking, so even a much worse retriever does not change which cause dominates.

If pushed on "so retrieval never matters": it does, and the repository measures the other side. The gap between BM25 and a seeded shuffle **reaches 0.9722**, **so the retriever** matters a great deal. The claim is about which lever moves further on this corpus, not that one of them is inert.

The overlap is a guarantee threshold, and it is exact

For uniform windows of size S advanced by stride T , an answer of length L sits inside some window wherever it falls if and only if L is at most S minus T plus one. At size four hundred and stride three hundred that is **101 characters at size four hundred**, and every one of **all 3899 positions** survives at that length while one character

longer already fails somewhere. Above the threshold survival is a lottery with computable odds: at 150 characters a chunking with no overlap keeps **0.652 of positions**, falling **down to 0.557 for a four hundred** character answer even at eight hundred character windows.

If pushed on "then more overlap is always better": no, and this is the part I would want to be asked about. Window starts are multiples of the stride, so a smaller stride is not a superset of a larger one. There is **1 case in this plan** where more overlap made the ceiling **falls by 0.0389** worse while adding **293 more chunks**. Above the guarantee threshold, overlap moves the lottery rather than winning it, and there is a constructed counterexample in the test suite so the claim does not rest on this corpus.

The ceiling bounds every retriever, and it is tight

An upper bound nobody attains is a weaker claim than it sounds, so the oracle is in the repository for exactly this purpose. It ranks by overlap with the answer and attains the ceiling on every chunking in the plan, with a **worst absolute difference of 0.0**.

If pushed on "you never ran a real encoder, so how do you know it bounds that too": because the bound is not a statement about retrievers. It is a statement about which chunks exist. No retriever can return a chunk that was never made, and the oracle shows the bound is reachable, so a dense encoder sits between the shuffle and the oracle like everything else. What the missing encoder does affect is the *retriever range* in the lever comparison, and that is disclosed rather than glossed.

The chunker is the bigger lever

Holding the corpus and the questions fixed, changing between the two real retrievers moves span complete recall by **at most 0.1972**, while changing the chunker moves it by **up to 0.7917**, which is **4 times as far**.

If pushed on "four times is suspiciously round": it is a ratio of two measured ranges and it is quoted as an upper estimate rather than a settled figure, because the retriever range covers two lexical methods. A dense encoder could plausibly widen it, which would shrink the ratio. That caveat is in the README and in ADR-003 rather than only in this answer.

Latency, if anyone asks

At the shipped corpus the **ceiling takes 2.2 ms** and retrieval is **22.5 times the ceiling** at the same size, over **7 timed repeats** on a two vCPU container running **Python 3.11.15**, measured **from 120 documents and out to 1920 documents** with **10558 chunks in the index**. There the ceiling is **37.3 ms at the largest** size while **retrieval takes 11974.3 ms**, and the ratio **widens to 320.7 times**. The ceiling has a **linearity ratio of 1.05** while **retrieval's is 14.949**. That widening is the useful direction: the argument for computing the ceiling first gets stronger with corpus size rather than weaker. The whole sweep of 13 chunkings by four retrievers by five values of *k* runs in **under 10 seconds**.

Questions that are meant to be hard

Is this just recall at k with extra steps? No, and the difference is stateable in one sentence: recall at *k* asks whether any retrieved chunk touches the answer, and this asks whether any single one holds it. What is mine beyond that: the containment ceiling and the proof it is attained, the three way failure attribution, the closed form for the guarantee threshold checked by exhaustion, the non monotonicity result with a constructed counterexample, and the receipts pipeline that fails the build when a document quotes a number the code no longer produces. Around a thousand statements of source, **233 tests, 99.2 percent line coverage**.

Your corpus is generated. Does any of this transfer? Two kinds of claim, and they transfer differently. The ceiling arithmetic and the guarantee threshold are properties of the chunker and transfer to any corpus. The

magnitudes do not: 0.2917 at two hundred characters is a property of these answer lengths. What a real corpus changes is the answer length distribution, and the whole point of the guarantee threshold is that a team computes their own number from theirs.

Why is there no transformer in a repository about chunking for RAG? Because the weights host was unreachable from the environment I built it in, so I wrote the path, fenced it behind an extra, tested it with the module made unimportable, and made the tool print that it has never been run. I was willing to ship without it because the ceiling bounds every retriever including that one, and the oracle shows the bound is attained. Offering this before being asked is worth more than defending it afterwards.

What is the weakest part? The corpus is generated, so every magnitude is a property of a corpus I wrote. Second weakest, and the more interesting answer: the retriever range covers two lexical methods, so the claim that the chunker is four times the lever is an upper estimate rather than a settled ratio.

Why should I trust the whole document row, which wins on your own metric? You should not treat it as the recommendation, and the repository says so in prose because the table cannot. It has a perfect ceiling and it ships **7662 characters per question**, against the best real alternative that **reaches 0.8278 on 3731 characters**. What those extra characters cost in generation quality, latency and token spend is not measured here, and that is in the section on what the repository cannot establish rather than buried.

Did anything go wrong while you built it? Four things, and all four are in the README under "Hardest problem solved" with the fix beside each. The useful one is the ceiling violation: a measured span complete recall of 0.850 came in against a ceiling of 0.828, which cannot happen. The cause was comparing character offsets across documents, since offsets are per document and a chunk covering characters 0 to 800 of one document numerically contains an answer at characters 100 to 500 of another. The invariant is now a test and it is the most valuable one in the repository, because it can only fail when a measurement is impossible.

How would you know if your corpus was too easy? By checking the direction rather than the magnitude, which is how I caught the version that was too *hard*. Eight topics and eight subjects repeat every sixty four documents, so a question naming only those two matched fifteen documents equally well and every retriever scored near chance while the attribution reported a retrieval problem. The tell was that it claimed retriever fixable failures on chunkings whose ceiling made that impossible.

What this repository cannot establish

Its own section, because it is the part a senior reviewer reads first, and offering it before being asked is worth more than any of the claims above.

- **A dense transformer encoder was never run.** The path is written, fenced behind an extra, and has produced no number. It does not threaten the ceiling, which bounds that retriever too, and it does narrow the retriever range in the lever comparison.
- **The corpus is generated.** What transfers is the ceiling arithmetic, which is a property of the chunker, and the guarantee threshold, which is a closed form.
- **The recursive splitter is a reimplement.** It reproduces documented behaviour and is not a port, so its numbers describe this implementation. That is deliberate: a chunker whose behaviour is a dependency's implementation detail makes every figure a statement about a version number.
- **Answer quality is not measured.** Containment is about retrievability. Whether a model handed a complete answer produces a good one needs a grader with its own error rate.
- **The downstream cost of long chunks is not measured.** The character count is priced and nothing beyond it.
- **Multi hop answers are outside the definition.** Containment is defined against one answer span in one document. A question needing two documents combined is not badly handled, it is out of scope, and saying

that is better than extending the metric to a case it would get wrong.

Things to say, and things not to say

Say:

- "the ceiling bounds every retriever, and the oracle shows it is attained."
- "1661 of 1663 failures were the chunker's, on this corpus, at k equal to 5."
- "the overlap is a guarantee threshold on answer length, and here is the closed form."
- "the chunker is the bigger lever, and that ratio is an upper estimate because I only ran lexical retrievers."
- "the ceiling costs milliseconds and retrieval costs seconds, so the ordering is not a preference."

Do not say:

- **"zero retrieval failures."** Say ten of thirteen chunkings had none over 360 questions, which supports **below 0.28 percent** and nothing stronger.
- **"chunking is more important than retrieval."** Say it is the bigger lever on this corpus, and that the gap to a seeded shuffle **reaches 0.9722, so the retriever** is not irrelevant.
- **"my hashed retriever is a dense encoder."** It hashes character n grams with inverse document frequency weighting. Nothing in the repository calls it neural, and neither should I.
- **"99.2 percent coverage means it is correct."** It means the lines ran. The ceiling invariant is the test that means something.
- **"the whole document chunker is the answer."** It wins on the one metric this repository measures and loses on the ones it does not, and the sentence has to carry that clause.

The live demo, five commands

```
# 1. The plan, and which retriever has never been run here.
python -m chunkaudit plan

# 2. A common chunk size, and what it destroyed. Exit 1.
python -m chunkaudit audit --size 200 --stride 200

# 3. The same corpus with whole document chunks. Exit 0, and see the cost.
python -m chunkaudit audit --strategy document

# 4. Every chunking, sorted worst ceiling first.
python -m chunkaudit sweep --html report.html

# 5. Every published figure re-measured and checked against the documents.
make receipts
```

If there is time for one more, `make bench` regenerates the latency table and `make charts` redraws the chart from its JSON, which is the shortest demonstration that no figure in either document was typed by hand.

Where to look in the code

Question	File
Why answers are character spans rather than strings	src/chunkaudit/documents.py, and ADR-002
The four chunking strategies, including the reimplemented recursive splitter	src/chunkaudit/chunking.py
The bound itself, with no model, no similarity and no ranking in it	src/chunkaudit/ceiling.py
The closed form for the guaranteed answer length	src/chunkaudit/ceiling.py, guaranteed_length
The two recalls and the three causes	src/chunkaudit/metrics.py
Which retrievers run and which one is written and never run	src/chunkaudit/retrieval.py, RUNNABLE and WRITTEN_NOT_RUN
The three verdicts and the exit codes they carry	src/chunkaudit/audit.py
Every threshold, with a comment on who chose it and how	configs/policy.yaml
The five decisions and their rejected alternatives	docs/adr/